OPCVM GSM

ARCHITECTURE DES BASES DE DONNÉES ET PERSISTANCE DES INTERFACES

Gestion sous mandat, Gestion libre, CRM, comité, ordres, liquidité et administration SGI

Objectif - Décrire la source de vérité, les modèles persistants, les flux de données et la stratégie de migration depuis les constantes/localStorage de la maquette vers PostgreSQL et les API Django/DRF multi-SGI.

Document d'architecture fonctionnelle et technique
Version 2.0 - 10 novembre 2026

État de référence de la maquette et du backend au 10 novembre 2026

1. Résumé exécutif

L'architecture actuelle est un monolithe modulaire Django/DRF couplé à un frontend React/Vite. PostgreSQL doit devenir la source de vérité des données métier ; Redis/Celery servent aux traitements asynchrones et au cache ; un stockage objet est prévu pour les documents. La migration est progressive : le frontend conserve encore certains jeux simulés et données localStorage, mais les modules backend structurants existent déjà.

> **Principe directeur**
> Une donnée métier ne doit avoir qu'une source de vérité. Les interfaces peuvent avoir plusieurs vues et caches, mais une position, un ordre, une décision de comité, une demande CRM ou un solde espèces ne doit pas exister sous plusieurs versions concurrentes.

2. Architecture logique cible

Couche	Technologie / composant	Responsabilité
Frontend	React + Vite + JSX	Interfaces Gestionnaire et Gestion libre, interactions utilisateur, visualisation, exports locaux transitoires.
API métier	Django + Django REST Framework	Validation, permissions, orchestration, transactions, multi-SGI, API versionnée.
Authentification	JWT + session Django Admin	Connexion API, sélection institution, accès Administration SGI par passerelle sécurisée.
Base métier	PostgreSQL	Comptes, institutions, clients, portefeuilles, ledger, ordres, comité, imports et CRM.
Données de marché	PostgreSQL (séparables logiquement/physiquement)	Indices, prix instruments, FX et référentiels de marché.
Cache / jobs	Redis + Celery	Cache, files de tâches, calculs asynchrones, notifications et verrous distribués si nécessaires.
Documents	S3 / MinIO / Blob (cible)	Imports Excel, KYC, contrats, avis d'opéré, rapports et pièces justificatives.

3. Multi-SGI et isolation des données

- Les modèles métier héritent d'InstitutionOwnedModel et portent institution_id.
- Le contexte institution est résolu par le backend à partir de X-Institution-ID et d'une InstitutionMembership active.
- InstitutionMembership définit les rôles ADMIN_SGI, HEAD_PM, PORTFOLIO_MANAGER, CUSTOMER_OFFICER, TRADER, BACK_OFFICE, COMPLIANCE, AUDITOR et CLIENT.
- Les viewsets doivent filtrer systématiquement leur queryset sur l'institution courante et injecter cette institution lors de la création.
- Les serializers doivent aussi vérifier que les clés étrangères référencent des objets du même tenant, sinon un utilisateur pourrait injecter une FK vers une autre SGI.

4. Modules persistants actuels

Module	Modèles principaux	Rôle
accounts	User	Identité applicative, email unique, langue et téléphone.
institutions	Institution, InstitutionMembership	Tenant SGI et habilitations.
instruments	Exchange, Issuer, Instrument	Référentiel titres et classes d'actifs.
clients	Client, KYCRecord, Mandate	Référentiel client, KYC et mandat.
portfolios	Portfolio, PortfolioAssignment, SecuritiesAccount, CashAccount	Portefeuilles, comptes et affectations.
ledger	SecurityMovement, CashMovement, PositionSnapshot, CashBalanceSnapshot, ValuationSnapshot, ForecastCashflow, WithdrawalRequest	Mouvements, positions, soldes, valorisations et retraits.
orders	Order, OrderEvent, Execution, Allocation	Ordres, historique, exécutions et allocations.
committees	CommitteeSession, CommitteeDecision, DecisionTarget, DecisionExecution, DecisionDeviation	Décisions d'investissement et mesure de leur exécution.
imports	ImportJob, ImportStagingRow	Réception, parsing, staging et future intégration atomique des fichiers.
marketdata	MarketIndex, IndexLevel, MarketPrice, FXRate	Historique de marché et conversions.
crm	Prospect, CRMActivity, CRMTask, CRMOpportunity, CRMRequest	Relation investisseurs et développement commercial.

5. Matrice de persistance des interfaces

Interface	Source actuelle	Source de vérité cible	État
Accueil / Vue Portefeuilles	Constantes + bootstrap Django + fallback local	clients, portfolios, ledger, marketdata	Transition
Money Management	Calculs frontend + données portefeuille	ledger + règles métier backend	Transition
Cession_Retrait	Maquette locale / workflow UI	WithdrawalRequest + ledger + ordres/transactions	À consolider
Ordres / Avis	Données maquette + modèles backend	orders + ledger + documents	Backend partiel
Rapport de comité	Décisions locales + compat bootstrap	committees + orders + marketdata	Backend disponible, UI à brancher complètement
Gestion libre / import SGI	localStorage et Excel côté navigateur	imports + portfolios + ledger	Import commit à finaliser
CRM	localStorage	crm + clients + institutions	Patch backend prêt, bridge API à finaliser

Interface	Source actuelle	Source de vérité cible	État
Administration SGI	Session Django après code d'attribution	Django Admin filtré tenant + permissions	Fonctionnel local
Documentation	Assets statiques public/documentation	CDN / objet ou fichiers statiques versionnés	Simple et déployable

6. Clients, KYC, mandats et CRM

- Client est le référentiel maître. Le CRM ne doit pas dupliquer les informations d'identité déjà portées par Client.
- KYCRecord reste lié en OneToOne au client ; les dates de revue/expiration peuvent générer des tâches CRM.
- Mandate distingue gestion sous mandat, gestion conseillée et gestion libre.
- CRMActivity, CRMTask, CRMOpportunity et CRMRequest peuvent pointer vers Client ; les objets pré-client utilisent Prospect.
- La conversion d'un Prospect en Client doit être une opération backend contrôlée, auditable et idempotente.

7. Portefeuilles, comptes et ledger

Objet	Règle de persistance
Portfolio	Identifié par code unique dans l'institution ; lié à Client et éventuellement Mandate.
SecuritiesAccount	Compte-titres interne ou externe, marché et devise.
CashAccount	Compte espèces par portefeuille/devise ; le solde ne doit pas être édité arbitrairement.
SecurityMovement	Écriture de titres avec quantité, prix, frais, taxes, dates et référence externe unique.
CashMovement	Écriture espèces datée ; sert à reconstituer les soldes.
PositionSnapshot	Vue/snapshot de position ; doit être cohérente avec les mouvements et lots.
ValuationSnapshot	Valeur du portefeuille à une date / devise.
ForecastCashflow	Coupons, dividendes, amortissements et flux attendus.

8. Ordres et exécutions

1 Création Order avec portefeuille, instrument, sens, type, quantité/montant et source.

↓

2 Contrôles mandat, cash/titres disponibles, limites et habilitations.

↓

3 Chaque changement de statut crée un OrderEvent.

↓

4 Les exécutions partielles/totales créent Execution puis Allocation par portefeuille lorsque nécessaire.

↓

5 Les mouvements titres/espèces sont inscrits dans le ledger avec transaction atomique.

↓

6 Les valorisations, positions, alertes et avis d'opéré sont recalculés / générés.

9. Comité de gestion

- CommitteeSession représente une séance ; CommitteeDecision une instruction d'investissement.
- Les prix min/max/limite doivent être null pour les types d'ordre où ils ne sont pas applicables et validés uniquement pour les ordres limite.
- DecisionTarget stocke la recommandation par portefeuille ; DecisionExecution relie l'exécution réelle et son résultat.
- DecisionDeviation impose une justification structurée des écarts entre décision et exécution.
- Les deux indices comparés dans l'interface doivent à terme être alimentés par MarketIndex / IndexLevel, pas par des séries simulées.

10. Imports Excel et staging

1 Upload du fichier et création ImportJob avec hash SHA-256.

↓

2 Parsing des feuilles et création de ImportStagingRow sans modifier les données officielles.

↓

3 Normalisation, validation de schéma, devises, identifiants, doublons et référentiels.

↓

4 Prévisualisation des anomalies et validation humaine si nécessaire.

↓

5 Commit atomique vers Client/Portfolio/Accounts/Ledger/Orders/ForecastCashflow/Valuation selon les feuilles.

↓

6 Archivage du fichier source et rapport d'import pour audit.

11. API et compatibilité frontend

Endpoint / mécanisme	Usage
/api/v1/auth/token/	Obtention JWT.
/api/v1/auth/token/refresh/	Rafraîchissement JWT.
X-Institution-ID	Sélection du tenant actif.

Endpoint / mécanisme	Usage
/api/v1/compat/bootstrap/	Bridge de compatibilité pour données attendues par le grand App.jsx.
/api/v1/admin-access/	Vérification du code d’attribution et génération d’un lien de passerelle Admin.
/admin/bridge/	Création de session Django Admin à durée courte.
/api/v1/crm/prospects/	Prospects CRM.
/api/v1/crm/activities/	Activités CRM.
/api/v1/crm/tasks/	Tâches CRM, complete/reopen.
/api/v1/crm/opportunities/	Opportunités CRM.
/api/v1/crm/requests/	Demandes clients CRM.

12. Sécurité, audit et intégrité

- JWT pour l'API et sessions sécurisées pour Django Admin ; cookies Secure et SameSite adaptés en production HTTPS.
- ALLOWED_HOSTS, CSRF_TRUSTED_ORIGINS et CORS_ALLOWED_ORIGINS doivent correspondre aux domaines de production.
- Le code d'attribution Administration SGI doit être hashé ; jamais stocké en clair dans le frontend.
- Les permissions métier doivent être appliquées côté backend, pas seulement masquées dans le menu React.
- Les opérations financières doivent être idempotentes lorsque l'appel peut être rejoué et atomiques lorsqu'elles modifient plusieurs écritures.
- Un audit métier structuré reste à généraliser au-delà des timestamps standards.

13. Déploiement local et domaine

- En local : Vite sur 127.0.0.1:5173 et Django sur 127.0.0.1:8001.
- En production, privilégier un même domaine logique avec reverse proxy : / pour le frontend, /api/ et /admin/ vers Django.
- Les variables VITE_* sont publiques dans le bundle : ne jamais y placer de mot de passe ou secret.
- Les fichiers de Documentation placés dans public/documentation sont copiés tels quels dans le build Vite.
- Django static/collectstatic concerne l'Admin ; les PDF du frontend restent des assets Vite sauf migration ultérieure vers object storage.

14. Feuille de route de persistance

Priorité	Travail
P0	Durcir tenancy/FK, permissions rôles, audit et configuration production.
P0	Brancher Vue Portefeuilles / Gestion libre sur CRUD Django et finaliser le commit d’import Excel.
P1	Brancher CRM frontend sur les endpoints Django et supprimer localStorage comme source de vérité.
P1	Brancher Rapport de comité sur CommitteeDecision/Execution/Deviation et MarketIndex.
P1	Ledger complet pour mouvements, positions, cash, CMP et

Priorité	Travail
	valorisations.
P2	Celery pour recalculs, notifications, imports lourds et documents.
P2	Object storage pour KYC, rapports, avis d’opéré et pièces.
P2	Observabilité, sauvegardes, PRA/PCA, tests de charge et sécurité.

15. Gaps à ne pas confondre avec une fonction production

État actuel
Le backend constitue un socle sérieux mais n'est pas encore production-ready : validation inter-tenant des FK à renforcer, permissions fines incomplètes sur certains viewsets, Client non encore systématiquement relié au compte utilisateur du portail, import commit incomplet, plusieurs écrans encore alimentés par données locales/simulées, Celery non branché aux workflows critiques.